

**Implementação de Detecção de Overflow em uma Unidade
Aritmética MIPS de 4 *bits***

Enzo Rocha Leite Diniz Ribas

8 de junho de 2026

Resumo

Este trabalho apresenta a implementação do mecanismo de detecção de overflow em uma Unidade Lógica e Aritmética (ULA) baseada na arquitetura MIPS. A ULA desenvolvida suporta as operações lógicas AND, OR e NOR, além das operações aritméticas ADD e SUB e da operação de comparação SLT (*Set Less Than*). O principal objetivo foi implementar e validar a lógica responsável por identificar situações de overflow durante operações aritméticas com números representados em complemento de dois. Os resultados obtidos confirmam a correta detecção de overflow nos casos esperados, sem geração de falsos positivos em operações lógicas.

Sumário

1	Introdução	3
2	Objetivos	3
2.1	Objetivo Geral	3
2.2	Objetivos Específicos	3
3	Fundamentação Teórica	3
3.1	Arquitetura MIPS	3
3.2	Operações Implementadas	4
3.3	Representação em Complemento de Dois	4
3.4	Overflow em Complemento de Dois	4
4	Metodologia	5
4.1	Definições Iniciais	5
4.2	Desenvolvimento das Operações Lógicas e Aritméticas	5
4.3	Implementação do Somador	5
4.4	Subtração e Operação SLT	5
4.5	Deteção de Overflow	5
4.6	Validação por Casos de Teste	6
5	Implementação	7
5.1	Interface da ULA	7
5.1.1	Entradas	7
5.1.2	Saídas	8
5.2	Lógica de Deteção de Overflow	9
6	Resultados	9
6.1	Caso 1: Overflow Positivo (ADD)	9
6.2	Caso 2: Overflow Negativo (ADD)	10
6.3	Caso 3: Soma Sem Overflow	11
6.4	Caso 4: Set On Less Than (SLT)	12
6.5	Resumo dos Resultados	14
7	Discussão	14
8	Conclusão	14

1 Introdução

A Unidade Lógica e Aritmética (ULA) é um dos componentes centrais de qualquer processador moderno, sendo responsável pela execução de operações matemáticas e lógicas sobre dados binários. Na arquitetura MIPS32 (*Microprocessor without Interlocked Pipeline Stages*), a ULA desempenha papel fundamental na execução de instruções do tipo R e em diversas operações realizadas ao longo do ciclo de execução de uma instrução.

Durante operações aritméticas com números inteiros com sinal, pode ocorrer uma condição denominada transbordamento ou *overflow*, que consiste em o resultado da operação exceder a faixa de representação disponível para o número de bits utilizado. Em um sistema de 32 bits com complemento de dois, essa faixa é $[-2^{31}, 2^{31} - 1]$. A não detecção de overflow compromete a integridade dos resultados e pode levar a comportamentos incorretos do programa.

Este trabalho descreve a implementação e validação de uma ULA de 4 bits compatível com a arquitetura MIPS estudada na disciplina de Arquitetura e Organização de Computadores, incluindo o mecanismo de detecção de overflow para as operações aritméticas de Adição, Subtração e *Set on Less Than*.

A implementação está disponibilizada no repositório do projeto no github. [1].

2 Objetivos

2.1 Objetivo Geral

Implementar e validar a detecção de overflow em uma ULA de 4 bits compatível com a arquitetura MIPS.

2.2 Objetivos Específicos

- Implementar as operações lógicas AND, OR, NAND e NOR.
- Implementar as operações aritméticas ADD e SUB.
- Implementar a operação SLT (*Set Less Than*).
- Desenvolver a lógica de detecção de overflow para operações aritméticas.
- Validar o funcionamento por meio de casos de teste documentados.

3 Fundamentação Teórica

3.1 Arquitetura MIPS

A arquitetura MIPS é uma arquitetura RISC (*Reduced Instruction Set Computer*) amplamente utilizada no ensino de arquitetura de computadores em função de sua simplicidade e organização modular. Sua ULA recebe dois operandos de 32 bits e um sinal de controle de 4 bits que seleciona a operação a ser executada, produzindo um resultado de 32 bits e um conjunto de sinais de status [2].

3.2 Operações Implementadas

A Tabela 1 apresenta as operações suportadas, seus códigos de controle e o comportamento esperado.

Tabela 1: Operações suportadas pela ULA

$\bar{A}$	$\bar{B}$	Op_1	Op_0	Operação	Descrição	Detecta Overflow
0	0	0	0	AND	Conjunção lógica bit a bit	Não
0	0	0	1	OR	Disjunção lógica bit a bit	Não
0	0	1	0	ADD	Soma aritmética com sinal	Sim
0	1	1	0	SUB	Subtração aritmética com sinal	Sim
1	1	0	1	NAND	Negação da operação OR	Não
1	1	0	0	NOR	Negação da operação OR	Não
0	1	1	1	SLT	<i>Set on Less Than</i>	Sim

3.3 Representação em Complemento de Dois

Em sistemas digitais, inteiros com sinal são representados em complemento de dois.

A subtração $A - B$ é implementada somando-se A ao complemento de dois de B :

$$A - B = A + (\bar{B} + 1)$$

onde $\bar{B}$ denota o complemento bit a bit de B . Isso permite reutilizar o somador para ambas as operações.

3.4 Overflow em Complemento de Dois

O overflow ocorre quando o resultado aritmético verdadeiro não é representável na faixa disponível. As condições são:

- **Overflow positivo:** dois operandos positivos produzem resultado com bit de sinal igual a 1 (resultado interpretado como negativo).
- **Overflow negativo:** dois operandos negativos produzem resultado com bit de sinal igual a 0 (resultado interpretado como positivo).

Formalmente, o overflow pode ser detectado comparando os bits de sinal dos operandos e do resultado:

$$\text{Overflow} = (A_{msb} = B'_{msb}) \wedge (\text{Result}_{msb} \neq A_{msb})$$

onde B' é B no caso de ADD, ou $\bar{B} + 1$ no caso de SUB. Uma forma equivalente, baseada nos *carries* do bit mais significativo, é:

$$\text{Overflow} = C_{in,msb} \oplus C_{out,msb}$$

onde $C_{in,msb}$ e $C_{out,msb}$ são, respectivamente, o carry de entrada e o carry de saída do somador no *most significant bit* [3].

4 Metodologia

4.1 Definições Iniciais

O desenvolvimento do projeto foi conduzido de forma incremental, iniciando-se pela definição da arquitetura da Unidade Lógica e Aritmética (ULA) e de suas interfaces de comunicação. Nessa etapa foram especificadas as entradas de dados, representadas pelos operandos de 4 bits, as saídas responsáveis pelo resultado das operações e os sinais de controle utilizados para selecionar a funcionalidade desejada. Essa definição inicial foi fundamental para garantir a integração adequada entre os diferentes módulos do sistema.

4.2 Desenvolvimento das Operações Lógicas e Aritméticas

Após a especificação das interfaces, foram implementadas as operações lógicas básicas suportadas pela ULA, incluindo AND, OR, NAND e NOR. Essas operações foram desenvolvidas utilizando portas lógicas combinacionais, permitindo a manipulação bit a bit dos operandos de entrada. A validação dessas funcionalidades constituiu uma etapa importante para assegurar o correto comportamento da unidade antes da implementação das operações aritméticas.

4.3 Implementação do Somador

Em seguida, foi desenvolvido o somador de 4 bits baseado no mecanismo de propagação de carry. Nesse modelo, cada estágio do somador recebe o carry gerado pelo estágio anterior, permitindo a realização de operações de soma entre os operandos. Embora apresente maior tempo de propagação em comparação com arquiteturas mais avançadas, esse tipo de implementação possui simplicidade estrutural e é amplamente utilizado para fins educacionais.

4.4 Subtração e Operação SLT

Com o somador concluído, foram implementadas as operações de subtração e de comparação SLT (*Set Less Than*). A operação de subtração foi realizada por meio da técnica de complemento de dois, aproveitando a estrutura do somador já desenvolvida. A operação SLT foi construída a partir da análise do resultado da subtração entre os operandos, permitindo determinar se um valor é menor que o outro de acordo com a representação binária adotada.

4.5 Detecção de Overflow

Posteriormente, foi desenvolvida a lógica responsável pela detecção de overflow nas operações aritméticas. Essa etapa constituiu o principal objetivo do projeto, uma vez que o overflow representa uma condição em que o resultado de uma operação excede a capacidade de representação disponível para a quantidade de bits utilizada. A lógica implementada analisa os bits de sinal dos operandos e do resultado, identificando corretamente situações de estouro durante operações de soma e subtração.

4.6 Validação por Casos de Teste

Por fim, foram elaborados e executados diversos casos de teste com o objetivo de validar o funcionamento da ULA. Os testes contemplaram tanto situações normais de operação quanto cenários específicos capazes de provocar overflow, permitindo verificar a corretude das operações implementadas e a confiabilidade do mecanismo de detecção desenvolvido.

5 Implementação

5.1 Interface da ULA

A ULA, ilustrada na Figura 1, possui a seguinte interface:

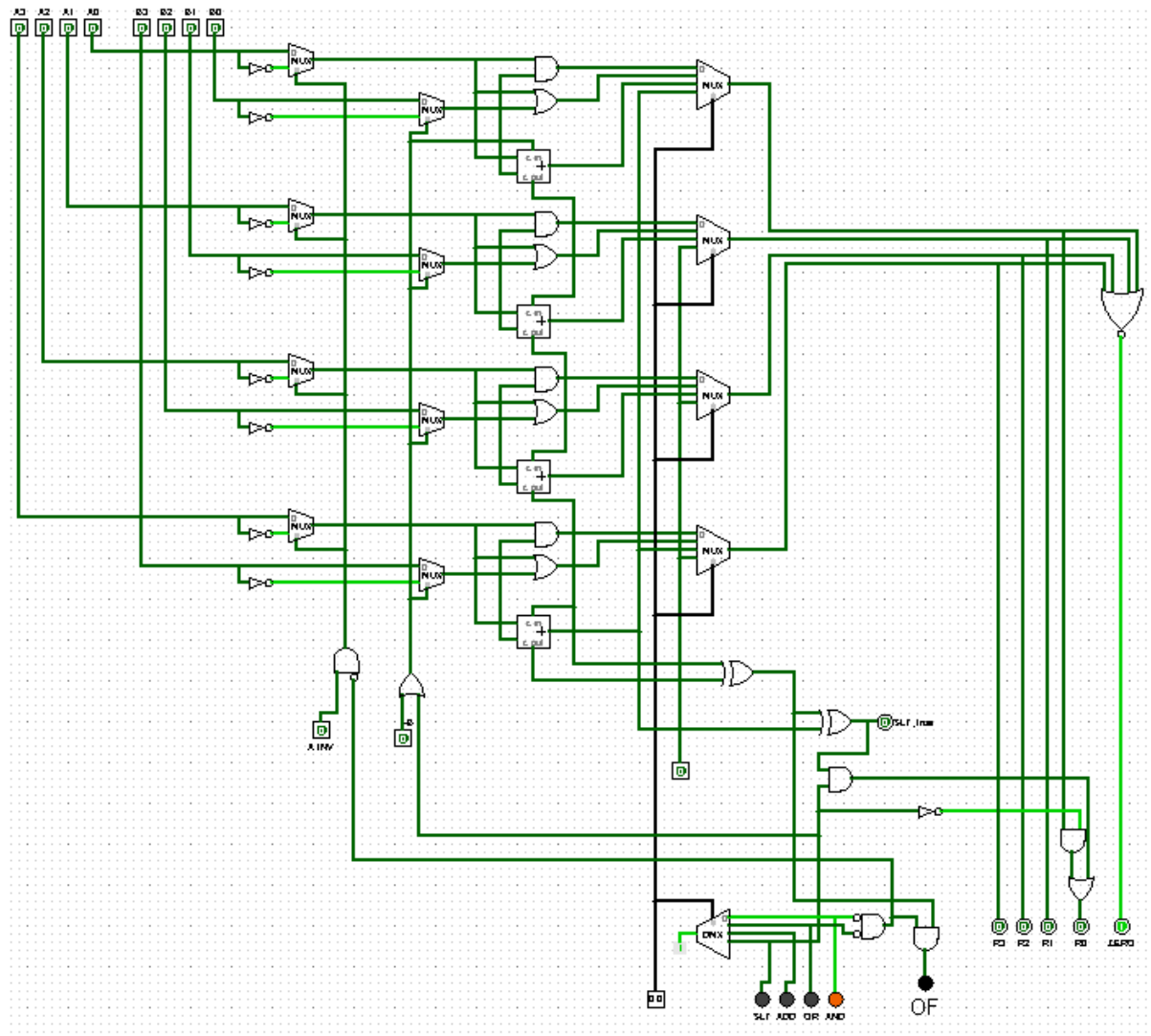


Figura 1: Interface da ULA de 4 bits. Fonte: Autor

5.1.1 Entradas

- A : Operando de 4 bits.
- B : Operando de 4 bits.
- InvertA: Sinal de controle para inverter A (1 bit).
- InvertB: Sinal de controle para inverter B (1 bit).
- Sel : Seletor de operação (2 bits).

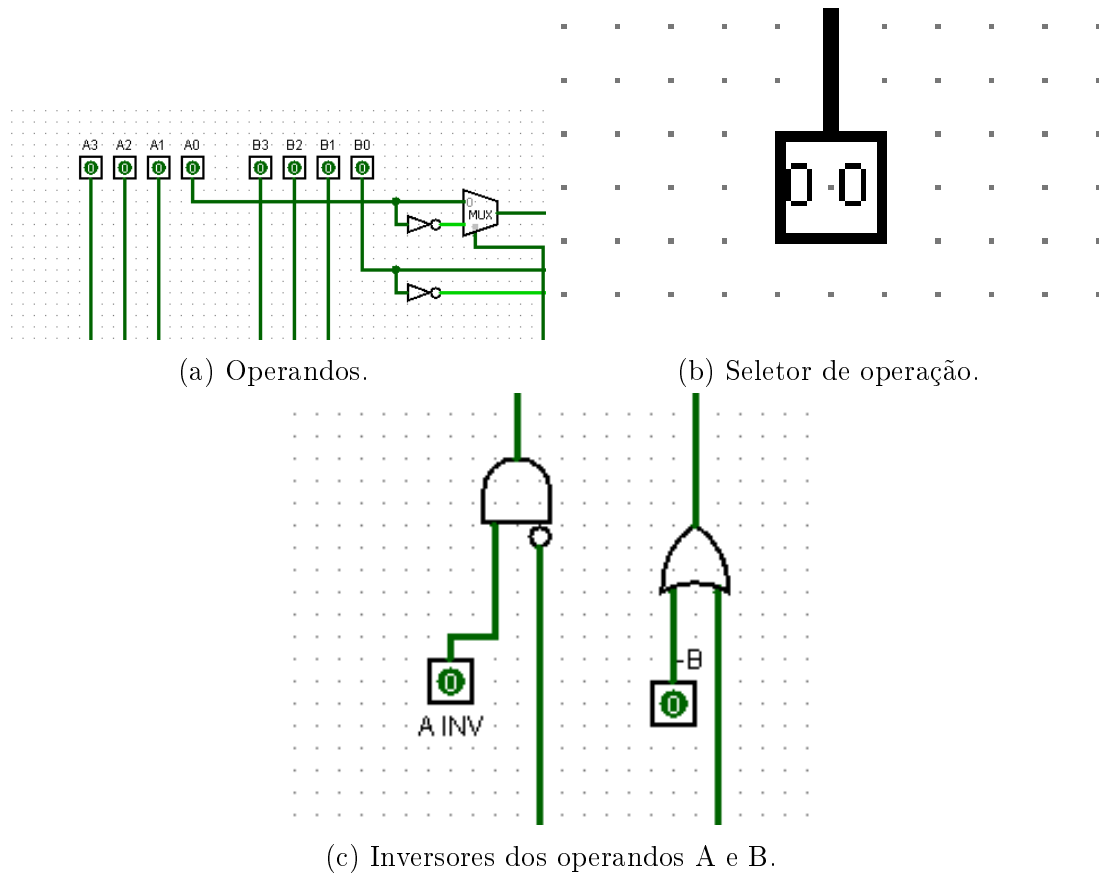


Figura 2: Entradas Fonte: Autor

5.1.2 Saídas

- *Result*: Resultado da operação (4 bits).
- *Zero*: Flag indicando se o resultado é zero (1 bit).
- *Overflow*: Flag indicando ocorrência de overflow (1 bit).
- *Operation*: Led indicando a operação realizada (1 bit).

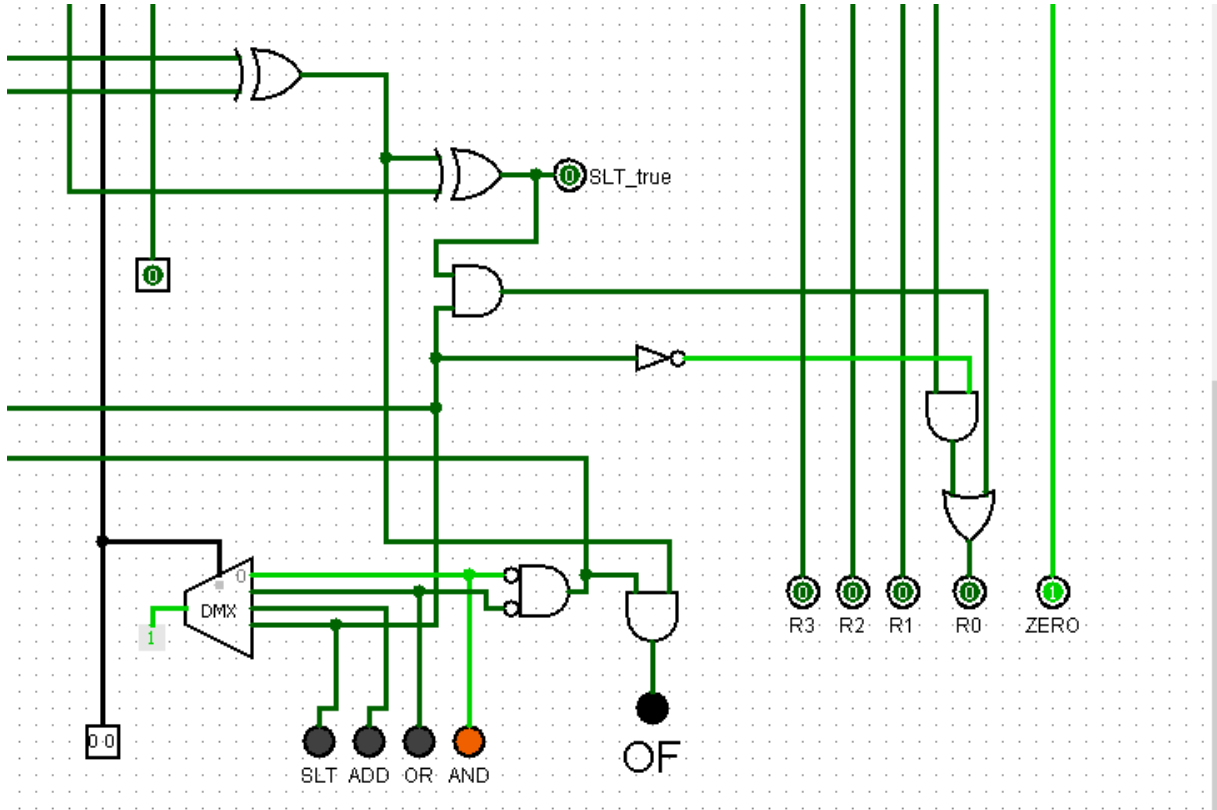


Figura 3: Saídas da ULA. Fonte: Autor

5.2 Lógica de Detecção de Overflow

A detecção de overflow foi implementada utilizando a abordagem baseada nos *carries* do bit mais significativo. A comparação entre o carry de entrada e o carry de saída do bit mais significativo do somador permite identificar corretamente as condições de overflow para operações de soma e subtração, conforme descrito na seção de fundamentação teórica.

$$\text{Overflow} = C_{in,msb} \oplus C_{out,msb}$$

A flag `Overflow` é habilitada apenas para ADD, SUB e SLT, permanecendo em nível lógico baixo para todas as operações lógicas.

6 Resultados

Foram realizados quatro casos de teste representativos, cobrindo overflow positivo, overflow negativo, operação sem overflow e operação lógica.

6.1 Caso 1: Overflow Positivo (ADD)

$$7 + 1 = -8$$

O resultado correto seria 8, que excede o valor máximo representável (7). O sinal de overflow foi corretamente ativado.

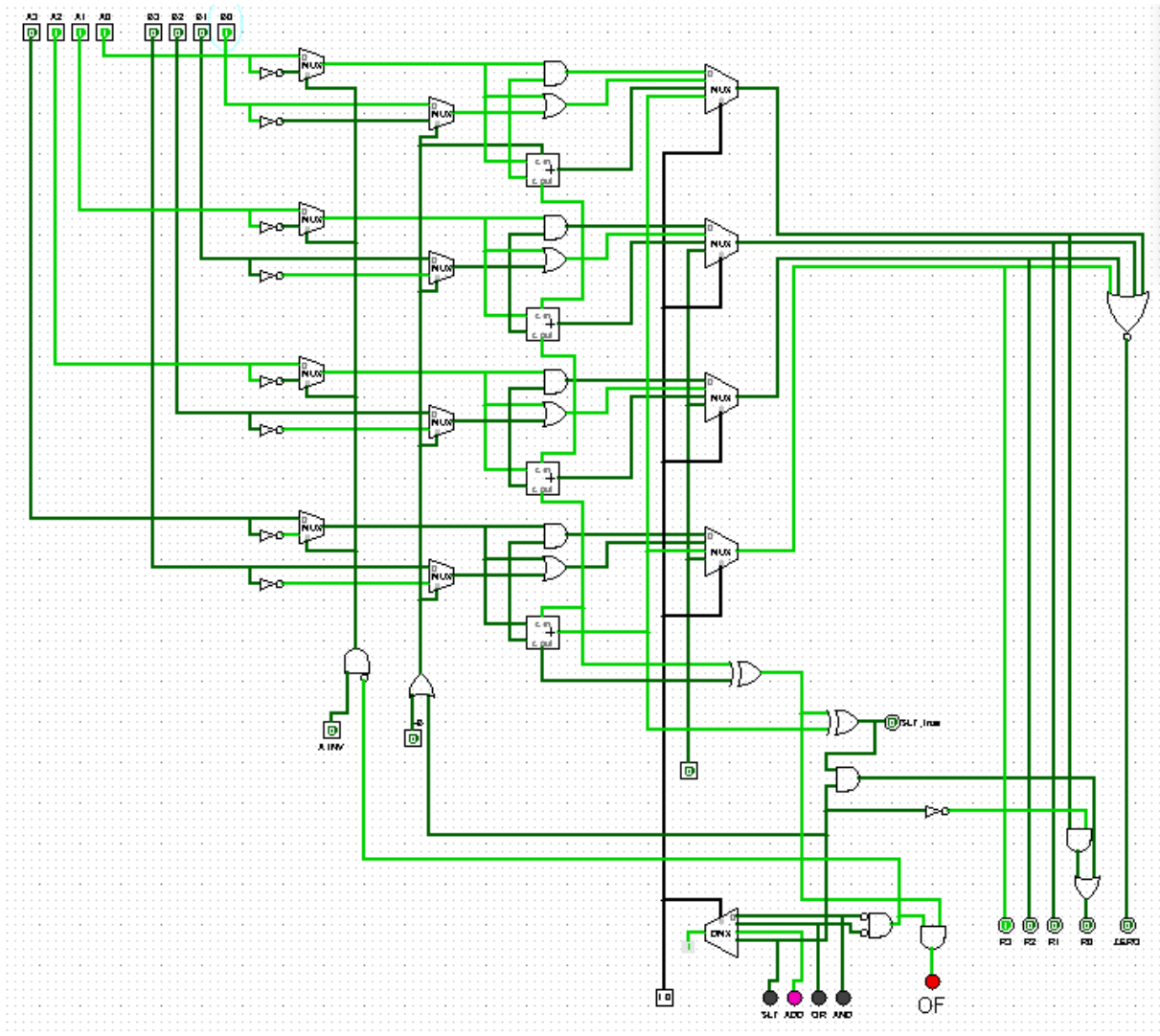


Figura 4: Caso 1: Overflow Positivo. Fonte: Autor

6.2 Caso 2: Overflow Negativo (ADD)

$$-7 - 2 = 7$$

O resultado correto seria -9 , abaixo do valor mínimo representável (-8). O sinal de overflow foi corretamente ativado.

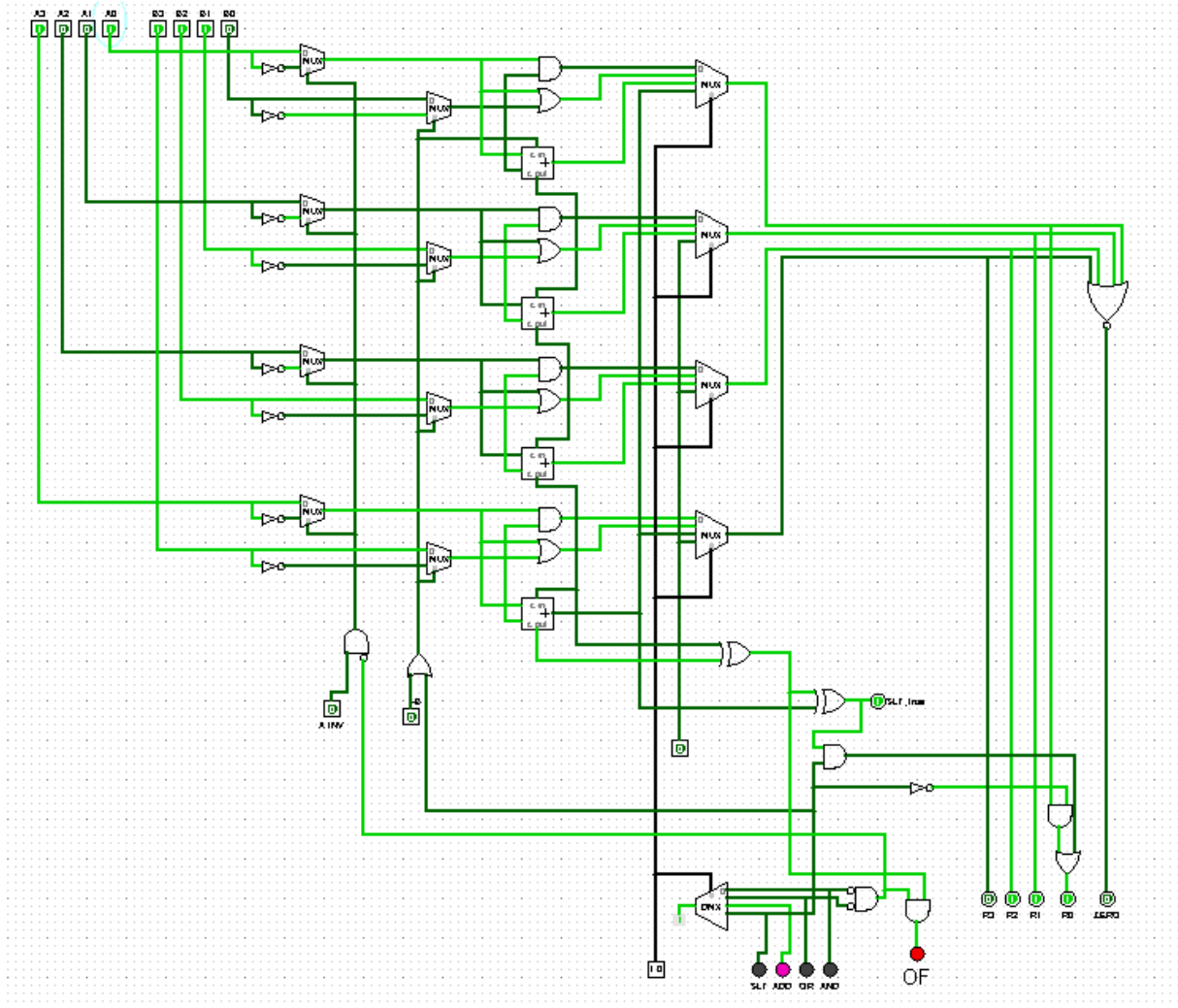


Figura 5: Caso 2: Overflow Negativo. Fonte: Autor

6.3 Caso 3: Soma Sem Overflow

$$1 + 5 = 6$$

O resultado está dentro da faixa representável. Overflow não foi ativado.

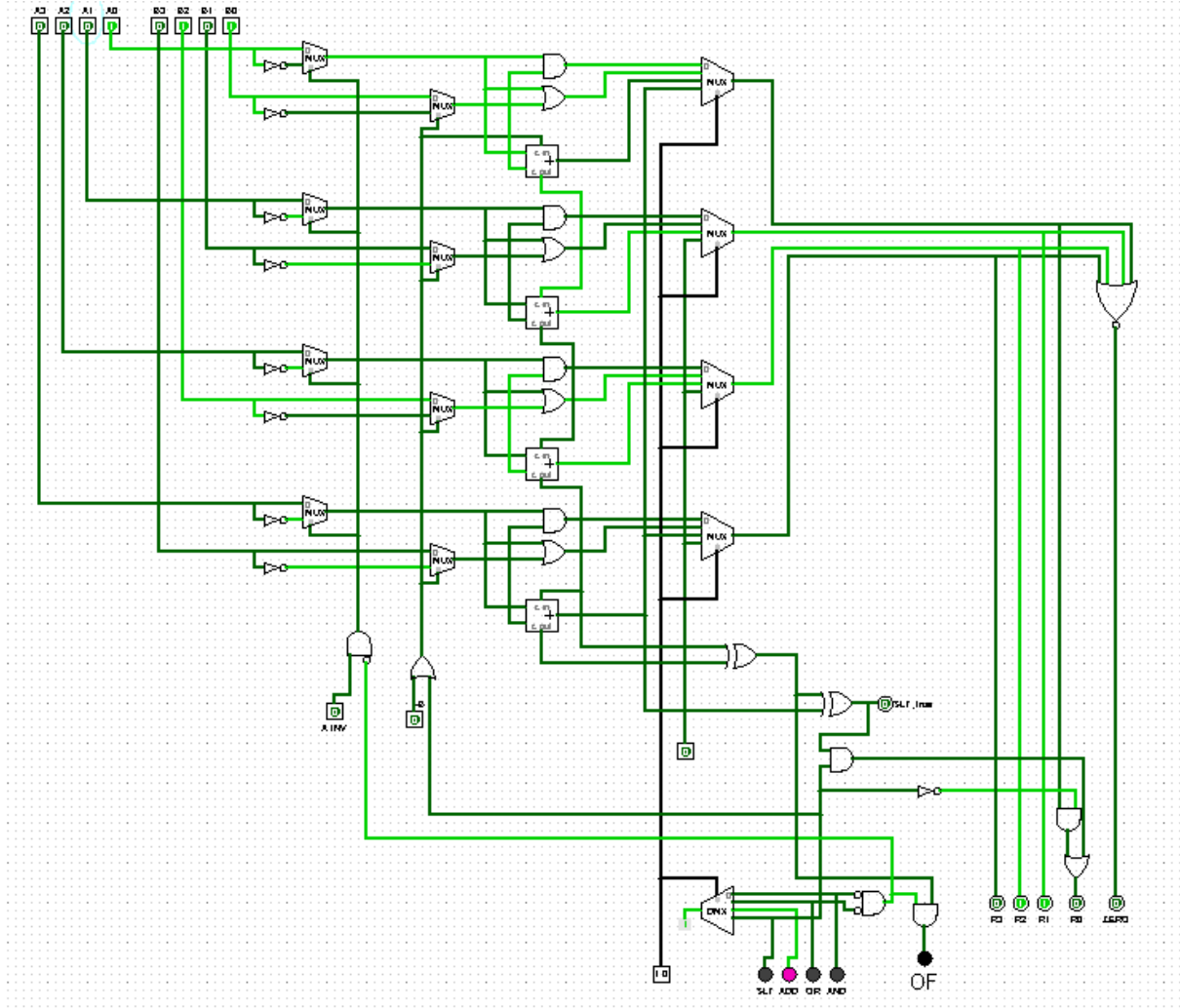


Figura 6: Caso 3: Soma Sem Overflow. Fonte: Autor

6.4 Caso 4: Set On Less Than (SLT)

A operação SLT compara A e B e define o resultado como 1 se $A < B$, ou 0 caso contrário. Ela é implementada por meio da subtração $A - B$ e análise do resultado. O overflow é relevante para SLT, pois um resultado incorreto devido a overflow pode levar a uma comparação errada. No caso testado, $A = -2$ e $B = 7$, portanto $A < B$ é verdadeiro, e o resultado esperado é 1. A operação $-2 - 7$ resulta em -9 , que é menor que zero, confirmando a condição de SLT. Entretanto o sinal de overflow é ativado, pois o resultado não é representável em 4 bits. Apesar disso, a lógica de SLT deve considerar o resultado da subtração e não apenas o overflow, garantindo que o resultado final seja correto (1) mesmo com overflow, mostrado na figura 7 pelo LED SLT_{TRUE} . Esse resultado é obtido pela operação:

$$SLT = OVERFLOW \oplus Result_{msb}$$

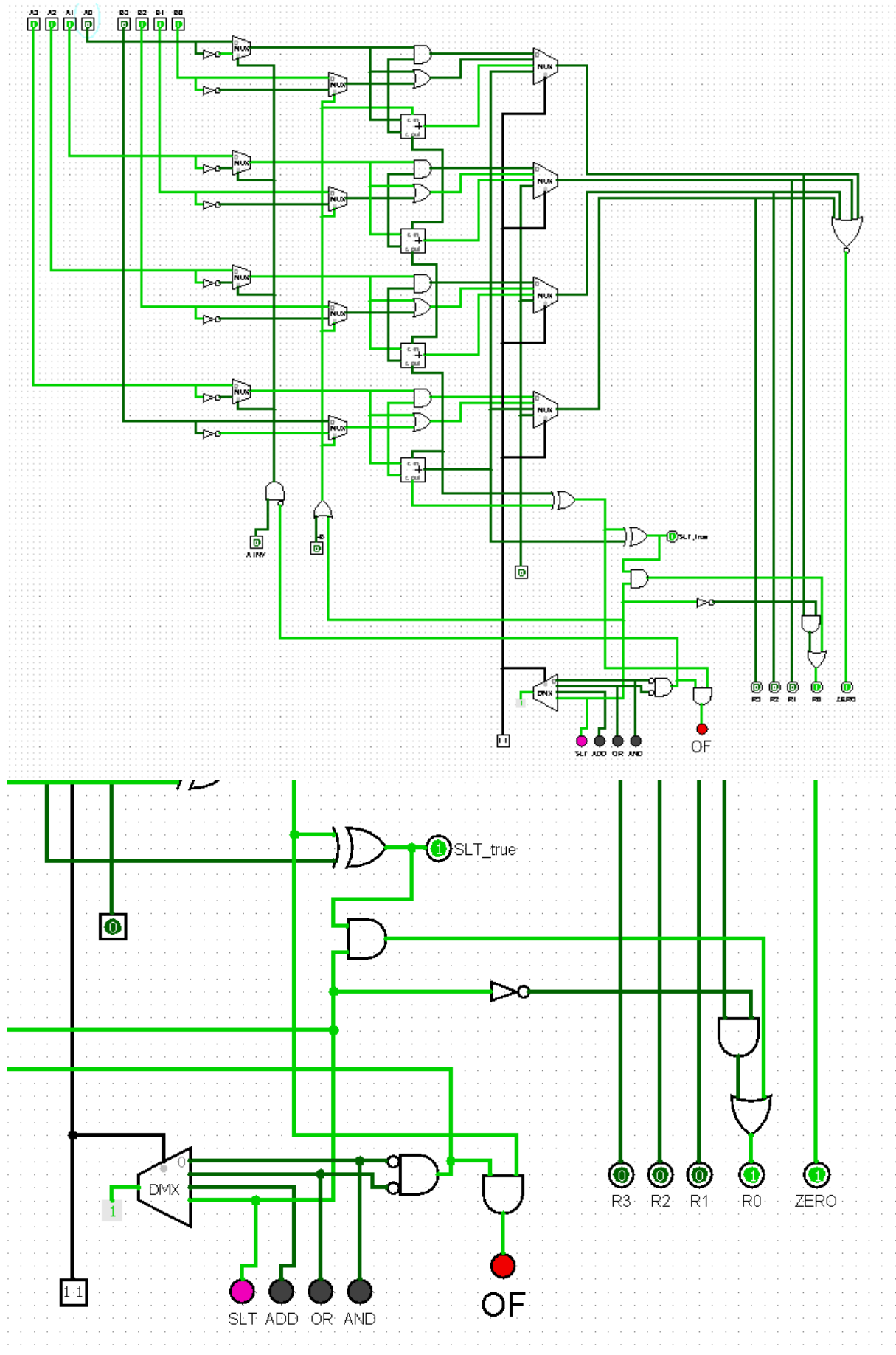


Figura 7: Caso 4: Set On Less Than. Fonte: Autor

6.5 Resumo dos Resultados

Tabela 2: Resumo dos casos de teste

Caso	Operação	Resultado Obtido	Overflow	Detectado Corretamente?
1	$7 + 1$	-8	Sim	Sim
2	$-7 - 2$	7	Sim	Sim
3	$1 + 5$	6	Não	Sim
4	$SLT(-2, 7)$	1	Sim	Sim

7 Discussão

Os resultados demonstram que o mecanismo implementado identifica corretamente situações de overflow em operações de soma, subtração e SLT, sem gerar falsos positivos em operações lógicas.

Vale reforçar que a operação SUB e SLT é implementada por complementação de B seguida de soma, reutilizando o caminho de dados do somador. Isso exige atenção na derivação da condição de overflow para subtração, uma vez que o operando B efetivo é $\overline{B} + 1$ e não B .

8 Conclusão

Neste trabalho foi implementada uma ULA de 4 bits baseada na arquitetura MIPS, suportando as operações AND, NAND, OR, NOR, ADD, SUB e SLT. A lógica de detecção de overflow foi desenvolvida com base na análise dos carries do bit mais significativo do somador, permitindo identificar corretamente situações de estouro aritmético durante operações de soma e subtração.

Os resultados obtidos confirmam o correto funcionamento da implementação: overflow é detectado nos casos de estouro aritmético e suprimido nas demais operações. O projeto permitiu consolidar conceitos fundamentais de arquitetura de computadores, representação numérica em complemento de dois e lógica digital.

Referências

- [1] E. R. L. D. Ribas, “Repositório do projeto - unidade lógica e aritmética com detecção de overflow.”
- [2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design MIPS Edition: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2017.
- [3] D. Harris and S. Harris, *Digital Design and Computer Architecture*, 3rd ed. Morgan Kaufmann, 2021.